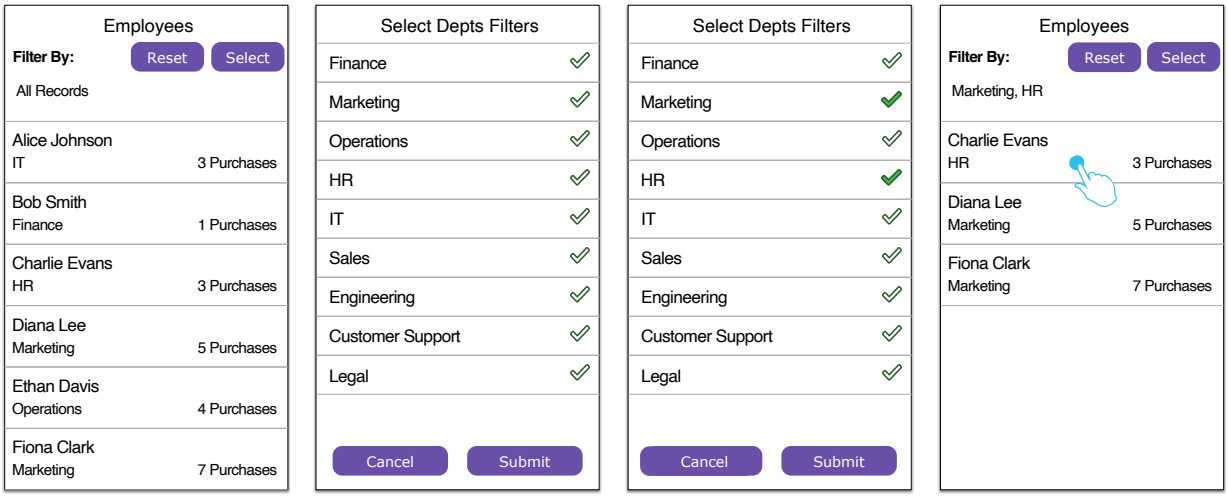
Mobile App Development
In-Class Assessment 2

Basic Instructions:

1. This is and In Class Assessment, which will count for 8% of the total course grade.
2. This assessment is an individual effort. Each student is responsible for her/his own assessment and its submission.
3. Once you have picked up the assessment, you may not discuss it in any way with anyone until the assessment period is over.
4. During the assessment, you are allowed to use the course videos, slides, and your code from previous home works and in class assignments. You can use the internet to search for answers. You are NOT allowed to use code provided by other students or solicit help from other online persons.
5. Answer all the assessment parts, all the parts are required.
6. During the assessment the teaching assistants and Instructors will pass by each student and ask them to demonstrate their application. Your interaction with the teaching assistants and instructors will be taken into consideration when grading your assessment submission.
7. Please download the support files provided with the assessment and use them when implementing your project.
8. Your assignment will be graded for functional requirements and efficiency of your submitted solution. You will loose points if your code is not efficient, does unnecessary processing or blocks the UI thread.
9. Create a zip file which includes all the project folder, any required libraries, and your presentation material. Submit the exported file using the provided canvas submission link.
10. **Do not try to use any Social Messenger apps, Emails, Or Cloud File Storage services in this exam.**
11. **Failure to follow the above instructions will result in point deductions.**
12. **Any violation of the rules regarding consultation with others will not be tolerated and will result disciplinary action and failing the course.**

In-Class Assessment 2 (100 Points)

In this assignment you will be building an application that uses RecyclerViews/ListViews to manage Employees and their purchases. This app is composed of one activity (Main Activity) and multiple fragments.



(a) Employees no filter (b) Select Dept Filter (c) Select Dept Filter (d) Employees Filtered

Figure 1, App Wireframe

Part 1, Employees Fragment (30 Points):

This fragment displays the list of Employees. The requirements are listed below:

1. This fragment should receive the ArrayList of Employees from the DataServices class and should display the list of Employees as shown in Figure 1(a).
   a. Each row item should display the employee name, department and total number of purchases.
2. Clicking on an employee row item should communicate with the Main Activity using an interface to:
   a. Replace the current fragment with the Purchases fragment and send it the selected Employee object.
   b. Push the current fragment on the back stack.
3. Clicking on the “Select” button should communicate with the Main Activity using an interface to:
   a. Replace the current fragment with the Select Filter Departments fragment.
   b. Push the current fragment on the back stack.
   c. Upon returning from the Select Filter Departments fragment, this fragment should receive a list departments. The displayed list of employees should be refreshed to only display employees from the selected departments. Update the TextView to indicate the departments selected. As shown in Figure 1(d).
4. Clicking on the “Reset” button should:
   a. Remove the filter and display all the employees. Refresh the displayed list to reflect no filter.
   b. Update the TextView to indicate “All Records”.

Part 2, Select Filter Departments Fragment (30 Points):

This fragment allows the user to select departments to filter by as shown in Fig 1(b). The requirements are listed below:

1. Get the list of departments by traversing all the users and retrieving the unique list of departments to be displayed.
2. Clicking on a row item:
   a. You need to track which rows have been previously selected and not selected.
   b. Clicking a not-selected row, should toggle the status of the row to selected and update the image to filled check box. Similarly, clicking a selected row, should toggle the status of the row to not-selected and update the image to not-filled check box.
3. Clicking “Submit” communicate with the Main Activity using an interface to:
   a. Identify the list of selected departments.
   b. Find the Employees fragment by tag, and send it the list of departments selected.
   c. Pop the back stack.
4. Clicking “Cancel” communicate with the Main Activity using an interface to:
   a. Pop the back stack.

Part 3, Purchases Fragment (30 Points):

The Purchases fragment displays the list of Purchases made by the selected Employee as shown in Fig 2(a). The requirements are listed below:

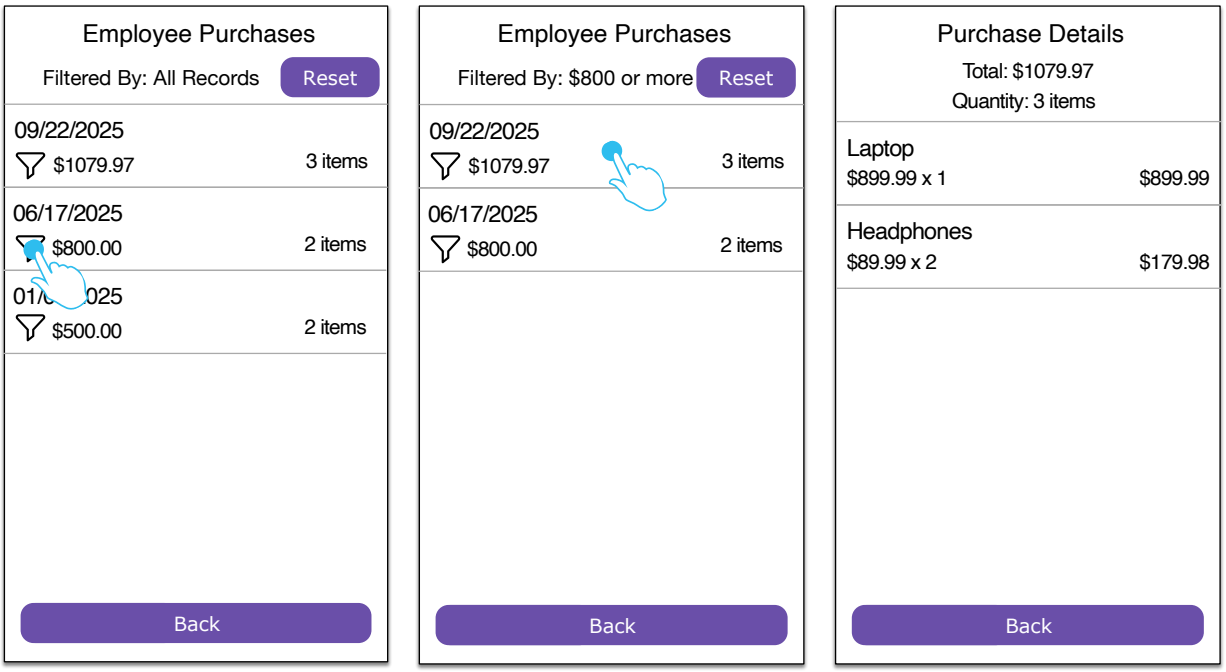
1. This fragment should receive the Employee object which contains the ArrayList of Purchases. Each row item should display:
   a. The purchase date.
   b. The total cost of the purchase list, computed as the sum of the (quantity x price) for each item in the purchase.
   c. The total number of items, computed as the total number of quantity in purchase.
2. Clicking on a row item should communicate with the Main Activity to:
   a. Replace the current fragment with the Purchase Details fragment and send it the selected Purchase object.
   b. Push the current fragment on the back stack.
3. Clicking on back button should communicate with the Main Activity to:
   c. Pop the back stack.
4. Clicking on the Filter icon on a row item should:
   a. Filter the displayed purchases to only include purchases that are at the selected total cost or more.
   b. Update the TextView to indicate the filter criteria as shown in Figure 2(b).
5. Clicking on the “Reset” button should:
   a. Remove the filter and display all the purchases. Refresh the displayed list to reflect no filter.
   b. Update the TextView to indicate “All Records”.

Part 4, Purchase Details Fragment (10 Points):

The Purchase Details fragment displays the Purchase details as shown in Fig 2(c). The requirements are listed below:

1. This fragment receives the Purchase object which contains the ArrayList of Items.
   a. Each row item should display:
      i. The item name
      ii. $item.price x item.quantity in both text and total as shown in Fig 2(c).
   b. The rows should be started by total cost in descending order.
2. At the top display the total cost and the total number of items.

3. Clicking on back button should communicate with the Main Activity to:
   a. Pop the back stack.



(a) Employees Purchases (b) Employees Purchases (c) Purchase Details

Figure 2, App Wireframe

Section:	
Student Name:	
Student ID:	

Part #	Features	Total	Grade	Comments
Part 1	- Displays the Employees as shown in Figure 1(a).	15		
Part 2	- Displays the departments as shown in Figure 1(b). - Account for the selected departments - Send the list of departments selected to the Employees screen, and displays the comma separated departments.	30		
Part 1	- Filter and reload the list of Employees based on the received departments array. - Reset functionality.	15		
Part 3	- Displays the Purchase objects as shown in Figure 2(a). - Calculates total and quantity correctly.	15		
Part 3	- Filter and reload the list of Purchases based on the selected total amount, and displays purchases that are at or above the selected amount. - Displays the filter criteria as shown in Figure 2(b). - Reset functionality.	15		
Part 4	- Displays the Items as shown in Figure 2(c) - Calculates and displays the total cost and quantity.	10		
	Total	**100**		

Table 1: Grading Key